Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет прикладної математики та інформатики

Кафедра програмування

Лабораторна робота №5

АЛГОРИТМ ФЛОЙДА

з курсу “Паралельні та розподілені обчислення”

Виконав:

студент групи ПМІ-33

Яценко Владислав Тарасович

Львів – 2025

Мета роботи
Реалізувати та дослідити **паралельний алгоритм Флойда–Варшалла** для знаходження найкоротших шляхів між усіма вершинами графа.
Порівняти швидкодію послідовної та багатопотокової реалізацій, визначити прискорення та ефективність залежно від кількості потоків.

Теоретичні відомості

Алгоритм Флойда–Варшалла
Алгоритм використовується для пошуку **найкоротших відстаней між усіма парами вершин** орієнтованого графа з вагами на ребрах (можливі додатні та нульові ваги, але не допускаються від'ємні цикли).
Основна формула оновлення відстаней:

$$dist[i][j] = \min(dist[i][j],\ dist[i][k] + dist[k][j])$$

де:

- i — початкова вершина;
- j — кінцева вершина;
- k — проміжна вершина (ітераційний параметр);
- dist[i][j] — поточна найкоротша відстань від i до j

Алгоритм має часову складність: $O(n^3)$, де n — кількість вершин у графі.

Паралелізація
Алгоритм паралелізується за **рядками матриці** (індекс i), вершини діляться на групи між потоками.
Зовнішній цикл по k залишається послідовним, оскільки кожна ітерація залежить від попередньої
Кожен потік обробляє свою частину даних, синхронізуючи результати.
Якщо кількість вершин не ділиться націло, "зайві" вершини обробляються останнім потоком.

Розподіл роботи між потоками(кожен потік обробляє свій діапазон рядків)**:**

$$rowsPerThread = \lceil \frac{n}{threadCount} \rceil$$

Реалізація програми

1. Послідовне виконання:

```
static void FloydWarshallSequential(int[,] dist, int[,] next)
{
    int n = dist.GetLength( dimension: 0);

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (i != j && dist[i, j] != INF)
            {
                next[i, j] = j;
            }
        }
    }

    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (dist[i, k] != INF && dist[k, j] != INF)
                {
                    int newDist = dist[i, k] + dist[k, j];
                    if (newDist < dist[i, j])
                    {
                        dist[i, j] = newDist;
                        next[i, j] = next[i, k];
                    }
                }
            }
        }
    }
```

2. Паралельне виконання:

```
static void FloydWarshallParallel(int[,] dist, int[,] next, int threadCount)
{
    int n = dist.GetLength( dimension: 0);

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (i != j && dist[i, j] != INF)
            {
                next[i, j] = j;
            }
        }
    }
```

```
for (int k = 0; k < n; k++)
{
    Thread[] threads = new Thread[threadCount];
    int rowsPerThread = (n + threadCount - 1) / threadCount;

    for (int t = 0; t < threadCount; t++)
    {
        int startRow = t * rowsPerThread;
        int endRow = Math.Min(startRow + rowsPerThread, n);
        int currentK = k;

        threads[t] = new Thread( start: () =>
        {
            for (int i = startRow; i < endRow; i++)
            {
                for (int j = 0; j < n; j++)
                {
                    if (dist[i, currentK] != INF && dist[currentK, j] != INF)
                    {
                        int newDist = dist[i, currentK] + dist[currentK, j];
                        if (newDist < dist[i, j])
                        {
                            dist[i, j] = newDist;
                            next[i, j] = next[i, currentK];
                        }
                    }
                }
            }
        });
        threads[t].Start();
    }
    foreach (Thread thread in threads)
    {
        thread.Join();
    }
}
```

Генерація графа

- Ймовірність існування ребра: 70%
- Ваги ребер: від 1 до 99
- Відсутність ребра: INF (нескінченність)
- Діагональ: 0

Результати експерментів

1. Граф 500×500 вершин

```
Введіть розмір графа (n): 500
Введіть максимальну кількість потоків: 12
Введіть початкову вершину (0-499): 8
Введіть кінцеву вершину (0-499): 487

Обчислення...

==================================================================================
||  Потоків    ||  Час виконання (мс)   ||   Прискорення   ||   Ефективність (%)   ||
==================================================================================
||           1 ||                  1179 ||          1,0000 ||               100,00% ||
==================================================================================
Найкоротша відстань від вершини 8 до 487: 5
Шлях (4 вершин): 8 -> 54 -> 117 -> 487

Деталі шляху з вагами:
Вершина 8 -> 54: вага = 2
Вершина 54 -> 117: вага = 1
Вершина 117 -> 487: вага = 2

Загальна відстань: 5 (сума всіх ваг)
==================================================================================
||  Потоків    ||  Час виконання (мс)   ||   Прискорення   ||   Ефективність (%)   ||
==================================================================================
||           2 ||                   803 ||          1,4682 ||                73,41% ||
||           3 ||                   603 ||          1,9552 ||                65,17% ||
||           4 ||                   544 ||          2,1673 ||                54,18% ||
||           5 ||                   545 ||          2,1633 ||                43,27% ||
||           6 ||                   582 ||          2,0258 ||                33,76% ||
||           7 ||                   531 ||          2,2203 ||                31,72% ||
||           8 ||                   622 ||          1,8955 ||                23,69% ||
||           9 ||                   605 ||          1,9488 ||                21,65% ||
||          10 ||                   647 ||          1,8223 ||                18,22% ||
||          11 ||                   668 ||          1,7650 ||                16,05% ||
||          12 ||                   693 ||          1,7013 ||                14,18% ||

=== Результати ===
Розмір графа: 500 × 500 вершин
Базовий час (1 потік): 1179 мс
Максимальна кількість потоків: 12

=== Найкращий результат ===
Оптимальна кількість потоків: 7
Час виконання: 531 мс
Прискорення: 2,2203x
Ефективність: 31,72%
```

Час виконання при 1 потоці – 1179мс.
Час виконання при 12 потоках – 693мс.

Аналіз результатів:

Характер залежності прискорення від кількості потоків:

1. **Зростання (2-7 потоки):**
   - ⇒ Прискорення монотонно зростає: 1.47x → 1.96x → 2.17x → 2.22x
   - ⇒ Найкраща ефективність при 2-3 потоках (73-65%)
   - ⇒ Оптимум досягається при 7 потоках (прискорення 2.22×)
2. **Деградація (8-12 потоки):**
   - ⇒ Прискорення падає: 1.90x → 1.95x → 1.82x → 1.77x → 1.70x
   - ⇒ Ефективність різко знижується: 23% → 14%
   - ⇒ Час виконання збільшується з 531 мс до 693 мс
3. **Причини обмеження продуктивності:**
   - ⇒ Накладні витрати на створення потоків.
   - ⇒ Синхронізація: Після кожної ітерації всі потоки чекають один одного (Threads.Join), що призводить до простою
   - ⇒ Невеликий обсяг роботи на потік: При 12 потоках кожен потік обробляє мало рядків, тобто час обчислень менший за час синхронізації

2. Граф 2000×2000 вершин

```
Введіть розмір графа (n): 2000
Введіть максимальну кількість потоків: 12
Введіть початкову вершину (0-1999): 192
Введіть кінцеву вершину (0-1999): 1471

Обчислення...

==========================================================================================
||  Потоків    ||  Час виконання (мс)   ||   Прискорення   ||   Ефективність (%)   ||
==========================================================================================
||           1 ||                 74299 ||          1,0000 ||               100,00% ||
==========================================================================================
Найкоротша відстань від вершини 192 до 1471: 3
Шлях (3 вершин): 192 -> 933 -> 1471

Деталі шляху з вагами:
Вершина 192 -> 933: вага = 1
Вершина 933 -> 1471: вага = 2

Загальна відстань: 3 (сума всіх ваг)
```

Потоків	Час виконання (мс)	Прискорення	Ефективність (%)
2	41682	1,7825	89,13%
3	28645	2,5938	86,46%
4	23550	3,1549	78,87%
5	24831	2,9922	59,84%
6	22223	3,3433	55,72%
7	19970	3,7205	53,15%
8	18525	4,0107	50,13%
9	18445	4,0281	44,76%
10	17205	4,3185	43,18%
11	16737	4,4392	40,36%
12	17168	4,3278	36,06%

```
=== Результати ===
Розмір графа: 2000 × 2000 вершин
Базовий час (1 потік): 74299 мс
Максимальна кількість потоків: 12

=== Найкращий результат ===
Оптимальна кількість потоків: 11
Час виконання: 16737 мс
Прискорення: 4,4392x
Ефективність: 40,36%
```

Час виконання при 1 потоці – 74299мс.
Час виконання при 12 потоках – 17168мс.

Аналіз результатів:

Характер залежності прискорення від кількості потоків:

1. Постійне зростання продуктивності (2-12 потоки):

- ⇒ Прискорення монотонно зростає від 1,78x до 4,44x
- ⇒ Відсутня деградація на великій кількості потоків
- ⇒ Час виконання постійно скорочується: 41682 мс → 16737 мс (при 11 потоках)
- ⇒ Навіть при 12 потоках зберігається високе прискорення 4,33x

2. Динаміка ефективності:

- ⇒ Найкраща ефективність при малій кількості потоків: 89% (2 потоки), 86% (3 потоки)
- ⇒ Поступове зниження ефективності зі збільшенням потоків: 78% → 59% → 55% → 53%

3. Оптимальні результати:

- ⇒ Максимальне прискорення: 4,44x при 11 потоках (час виконання 16737 мс)
- ⇒ Оптимальний час виконання: 16737 мс

4. Причини ефективного масштабування:

- ⇒ **Великий обсяг обчислень:** Граф розміром 2000×2000 вершин забезпечує достатньо роботи для кожного потоку
- ⇒ **Локальність даних:** Кожен потік працює зі своїм діапазоном вершин, зменшуючи конфлікти доступу до пам'яті

5. Обмеження масштабування:

- ⇒ Ефективність знижується через накладні витрати на управління потоками
- ⇒ Конкуренція за спільні ресурси (кеш, пам'ять) при великій кількості потоків
- ⇒ Закон Амдала: послідовна частина алгоритму обмежує теоретичне прискорення

Висновок

Паралельна реалізація алгоритму Флойда дозволяє скоротити час виконання у кілька разів на великих графах.
Проте ефективність зменшується при надмірній кількості потоків через синхронізаційні витрати.
Розподіл навантаження між потоками є критичним для отримання стабільного прискорення.